

ZKP Authentication Demo — Schnorr + Fiat-Shamir + PKI

Application pédagogique interactive illustrant l'authentification par **Zero-Knowledge Proof (ZKP)** avec le protocole de **Schnorr**, l'heuristique de **Fiat-Shamir**, une **PKI simulée** et la **simulation de 4 attaques cryptographiques**.

Table des matières

1. [Aperçu](#)
2. [Démonstration](#)
3. [Installation](#)
4. [Architecture du projet](#)
5. [Cryptographie expliquée](#)
 - [Zero-Knowledge Proof \(ZKP\)](#)
 - [Protocole de Schnorr](#)
 - [Heuristique de Fiat-Shamir](#)
 - [PKI dans ce projet](#)
6. [Simulation d'attaques](#)
7. [Routes API](#)
8. [Paramètres cryptographiques](#)
9. [Limites pédagogiques](#)

Aperçu

Ce projet est une démonstration **visuelle et interactive** des mécanismes qui permettent à un utilisateur de **prouver qu'il connaît un secret (son mot de passe) sans jamais le révéler** — ni sur le réseau, ni dans la base de données du serveur.

Il combine deux grandes familles de cryptographie :

Technologie	Rôle dans ce projet
ZKP / Schnorr	Authentifier l'utilisateur sans transmettre son secret
PKI (CA simulée)	Certifier que la clé publique Y appartient bien à un utilisateur donné
Fiat-Shamir	Rendre le protocole non-interactif (pas de tour-par-tour avec le serveur)
DLP 2048 bits	Fondement mathématique de la sécurité (RFC 3526, Groupe 14)

Démonstration

L'interface propose trois modes :

- **Inscription** : génération de clés + émission de certificat PKI en 6 étapes animées

- **Connexion** : preuve ZKP complète en 7 étapes détaillées
 - **Attaques** : 4 scénarios d'attaque simulés avec explication de pourquoi chacun échoue
-

Installation

```
# 1. Cloner le dépôt
git clone https://github.com/votre-pseudo/zkp-demo.git
cd zkp-demo

# 2. (Optionnel) Environnement virtuel
python -m venv venv
source venv/bin/activate      # Linux / macOS
venv\Scripts\activate        # Windows

# 3. Installer les dépendances
pip install -r requirements.txt

# 4. Lancer l'application
python app.py

# 5. Ouvrir dans le navigateur
# http://127.0.0.1:5000
```

Dépendances (`requirements.txt`) :

```
flask
cryptography
```

Note : Au premier démarrage, un fichier `pki_store.json` est automatiquement créé. Il contient la paire de clés RSA-2048 de la CA et tous les certificats émis. Ne pas versionner ce fichier (ajoutez-le à `.gitignore`).

Architecture du projet

```
zkp-demo/
├── app.py                # Backend Flask – toute la cryptographie
├── requirements.txt
├── pki_store.json        # Généré au démarrage (ne pas versionner)
├── static/
│   ├── app.js           # Appels API + rendu animé des étapes
│   └── style.css         # Thème clair pour présentation
└── templates/
    └── index.html        # Interface principale
```

Flux de données

```

Navigateur (Client)
|
| POST /register → { username, Y }          (jamais le password)
| POST /login   → { username, R, s, msg }   (jamais x ni password)
| POST /attack/* → { username }
|
Flask (app.py)
|
|— keygen(password)      x = SHA256(pwd) mod Q ; Y = G^x mod P
|— _issue_certificate()  CA signe { username, Y } → certificat RSA
|— _verify_certificate() Vérification signature CA
|— verify_zkp(Y, R, s, msg) G^s × Y^c ≡ R (mod P) ?

```

Cryptographie expliquée

Zero-Knowledge Proof (ZKP)

Un **Zero-Knowledge Proof** est un protocole cryptographique dans lequel un **prouveur** (Prover) convainc un **vérificateur** (Verifier) qu'il connaît une information secrète, **sans révéler aucune information sur ce secret**.

Pour être valide, un ZKP doit satisfaire trois propriétés :

Propriété	Définition	Dans ce projet
Complétude	Un prouveur honnête convainc toujours le vérificateur	Si le mot de passe est correct, la vérification Schnorr réussit toujours
Solidité (Soundness)	Un prouveur malhonnête ne peut convaincre qu'avec probabilité négligeable	Sans connaître x, la probabilité de forger une preuve valide est $1/Q \approx 2^{-2047}$
Zéro connaissance	Le vérificateur n'apprend rien sur le secret	Le serveur ne voit jamais x ni le mot de passe

Protocole de Schnorr

Le protocole de Schnorr est un protocole d'identification à connaissance nulle basé sur la **difficulté du logarithme discret (DLP)**.

Paramètres publics

```

P : nombre premier de 2048 bits (RFC 3526, Groupe 14)
Q : (P-1)/2 — ordre du sous-groupe
G : 2       — générateur

```

Ces paramètres sont connus de tous. La sécurité repose sur le fait que si l'on connaît $Y = G^x \bmod P$, retrouver x est computationnellement infaisable.

Phase 1 — Inscription (keygen)

```
Client :
  x = SHA256(password) mod Q    ← clé privée (jamais transmise)
  Y = G^x mod P                 ← clé publique

Transmission : username + Y → Serveur
Serveur stocke : { username: Y } (aucun mot de passe, aucun hash)
```

Phase 2 — Authentification (protocole complet)

```
Client :
  r = random ∈ [1, Q-1]        ← nonce secret à usage unique
  R = G^r mod P                 ← commitment
  c = SHA256(Y || R || session_id) mod Q ← challenge (Fiat-Shamir)
  s = (r - c·x) mod Q           ← réponse

Transmission : { R, s, session_id } → Serveur (x et r JAMAIS envoyés)

Serveur :
  c' = SHA256(Y_stockée || R || session_id) mod Q
  Vérifie : G^s × Y^c' ≡ R (mod P) ?
```

Pourquoi la vérification fonctionne

L'équation se vérifie algébriquement :

```
G^s × Y^c
= G^(r - c·x) × (G^x)^c
= G^(r - c·x) × G^(c·x)
= G^(r - c·x + c·x)
= G^r
= R ✓
```

Le serveur confirme que le client connaît x sans jamais voir ni x ni r .

Heuristique de Fiat-Shamir

Dans le protocole de Schnorr interactif original, le challenge c est envoyé **par le serveur** après réception de R . Cela nécessite deux allers-retours réseau.

L'heuristique de **Fiat-Shamir** transforme ce protocole en protocole **non-interactif** : le client calcule lui-même **c** en appliquant une fonction de hachage sur le contexte :

$$c = \text{SHA256}(Y \parallel R \parallel \text{message}) \bmod Q$$

Tant que SHA256 se comporte comme un **oracle aléatoire**, ce raccourci est cryptographiquement équivalent. C'est la base des **signatures numériques de Schnorr** (standardisées dans Bitcoin via BIP-340 et dans EdDSA).

Avantage : un seul aller-retour réseau. Le client envoie directement **(R, s, message)** sans attendre un challenge du serveur.

PKI dans ce projet

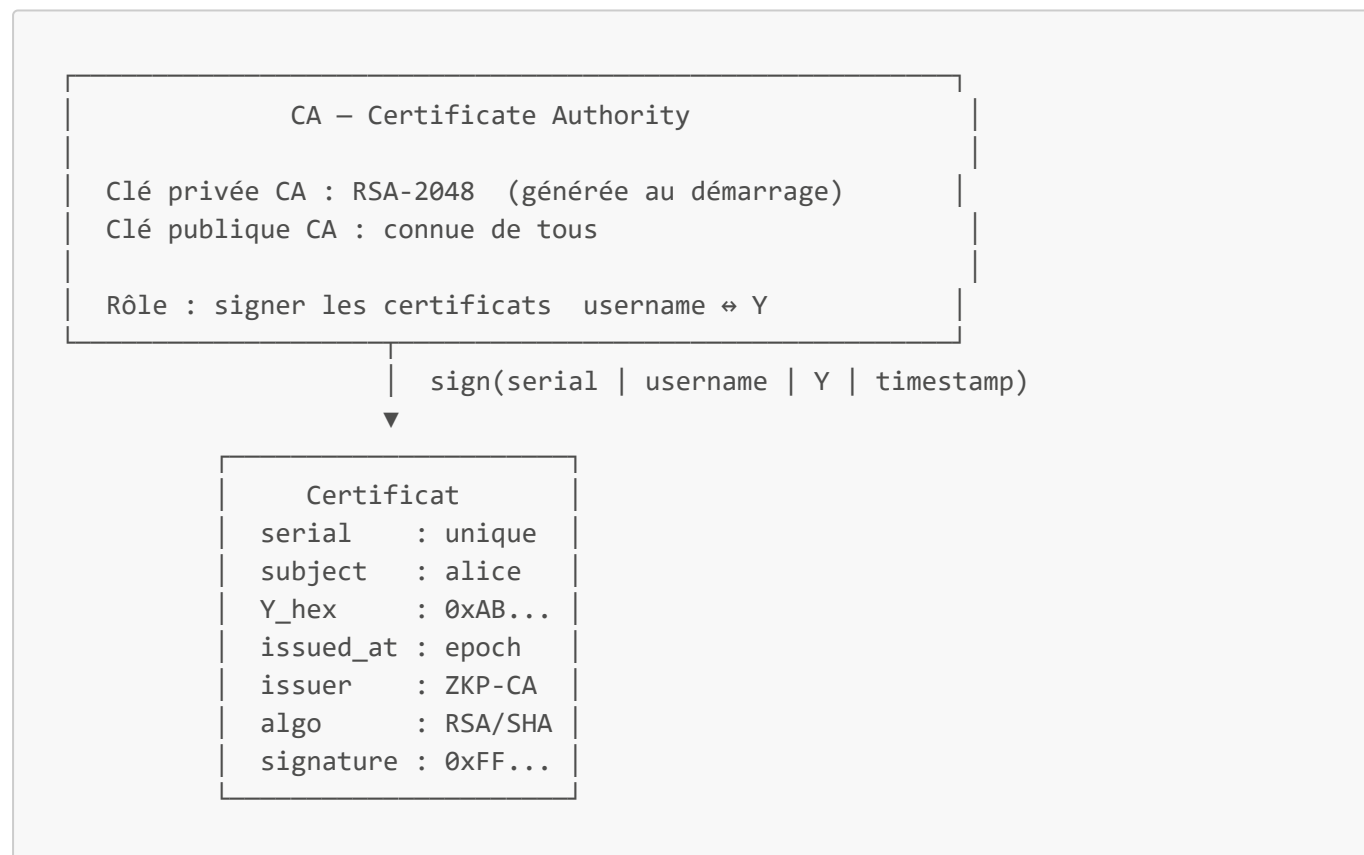
Problème résolu par la PKI

Sans PKI, un attaquant pourrait :

1. **Substituer sa propre clé publique Y'** à la place de Y lors de l'inscription
2. Se faire authentifier à la place de la vraie Alice

La PKI lie cryptographiquement une **identité** à une **clé publique**, grâce à une autorité de confiance (CA).

Architecture PKI simulée



Ce que fait la CA à chaque inscription

```
# Payload signé par la CA
payload = f"{serial}|{username}|{Y_hex}|{issued_at}".encode()

# Signature RSA-2048 / SHA-256
signature = ca_private_key.sign(payload, PKCS1v15(), SHA256())
```

Ce que fait le serveur à chaque connexion

Avant de vérifier la preuve ZKP, le serveur vérifie le certificat :

```
# Reconstitution du payload
payload = f"{serial}|{username}|{Y_hex}|{issued_at}".encode()

# Vérification signature CA
ca_public_key.verify(signature, payload, PKCS1v15(), SHA256())
# → Si la vérification échoue : connexion rejetée immédiatement
```

Pourquoi c'est important

Scénario	Sans PKI	Avec PKI
Attaquant substitue Y' à Y	✅ Possible — le serveur accepte Y'	❌ Impossible — la signature CA est invalide pour Y'
Attaquant forge un certificat	—	❌ Impossible sans la clé privée CA (RSA-2048)
Vol de la base de données	Y exploitable si on peut substituer	Y inutilisable sans certificat valide

Fichier pki_store.json

```
{
  "ca_private_key": "-----BEGIN RSA PRIVATE KEY-----\n...",
  "ca_public_key": "-----BEGIN PUBLIC KEY-----\n...",
  "users": {
    "alice": "0xABC123..."
  },
  "certificates": {
    "alice": {
      "serial": "a3f9e12b",
      "subject": "alice",
      "Y_hex": "0xABC123...",
      "issued_at": 1715000000,
      "algorithm": "RSA-2048 / SHA-256",
      "issuer": "ZKP-Demo CA",
      "signature": "0xDEF456..."
    }
  }
}
```

```

    }
  }
}

```

⚠ Dans un système réel, la clé privée CA serait stockée dans un **HSM** (Hardware Security Module) et jamais exposée en clair.

Simulation d'attaques

1. Mauvais mot de passe — [/attack/wrong-password](#)

L'attaquant ne connaît pas le mot de passe. Il génère un x' et un Y' différents du vrai x et Y .

```

x' = SHA256("ATTAQUE_xxxx") mod Q    ← différent du vrai x
Y' = G^x' mod P                      ← différent du Y stocké

Preuve envoyée : G^s' × Y'^c ≡ R'    (vraie pour Y', fausse pour Y_stockée)
Vérification   : G^s' × Y_stockée^c ≢ R' ← ÉCHEC

```

Défense : l'équation de Schnorr ne se vérifie que si le prouveur utilise le même x que celui qui a généré le Y stocké.

2. Replay Attack — [/attack/replay](#)

L'attaquant capture un paquet (R , s , $message$) valide sur le réseau et tente de le renvoyer.

```

Session capturée   : session_abc123 → c1 = SHA256(Y || R || session_abc123)
Nouvelle session   : session_def456 → c2 = SHA256(Y || R || session_def456)

c1 ≠ c2 → G^s × Y^c2 ≢ R ← ÉCHEC

```

Défense : le `session_id` change aléatoirement à chaque tentative. Le challenge c est lié cryptographiquement au message — une preuve valide pour une session est invalide pour toute autre.

3. Forgery — [/attack/forgery](#)

L'attaquant génère R et s complètement aléatoires, espérant que l'équation se vérifie par chance.

```

Probabilité de succès : 1/Q ≈ 1/22047 ≈ 0

```

Défense : la propriété de **soundness** du protocole de Schnorr garantit mathématiquement qu'un prouveur sans x ne peut réussir qu'avec une probabilité négligeable. Ce résultat est prouvé par réduction au DLP.

4. Man-in-the-Middle (MitM) — `/attack/mitm`

L'attaquant intercepte le paquet réseau (`R`, `s`, `message`) et tente d'en extraire `x`.

Tentative d'extraction de `x` :

$$s = r - c \cdot x \mod Q$$
$$\rightarrow x = (r - s) / c \mod Q$$

Problème : `r` n'est jamais transmis sur le réseau → extraction impossible

Tentative de substitution de `Y` :

Si l'attaquant remplace `Y` par `Y'` :

- Le certificat PKI ne correspond plus (signé pour l'ancienne `Y`)
- La vérification CA échoue → connexion rejetée

Double défense : ZKP empêche l'extraction du secret, PKI empêche la substitution de clé.

Routes API

Méthode	Route	Description
GET	<code>/</code>	Interface principale
POST	<code>/register</code>	Inscription — keygen + émission certificat PKI
POST	<code>/login</code>	Connexion — vérification PKI + preuve ZKP
GET	<code>/db</code>	Contenu de la base (clés publiques uniquement)
GET	<code>/pki</code>	Clé publique CA + tous les certificats émis
POST	<code>/attack/wrong-password</code>	Simulation mauvais mot de passe
POST	<code>/attack/replay</code>	Simulation replay attack
POST	<code>/attack/forgery</code>	Simulation forgery
POST	<code>/attack/mitm</code>	Simulation Man-in-the-Middle

Paramètres cryptographiques

Paramètre	Valeur	Référence
Groupe DH	RFC 3526 Groupe 14	2048 bits, P premier sûr
Générateur G	2	Standard

Paramètre	Valeur	Référence
Ordre Q	$(P-1)/2$	Sous-groupe d'ordre premier
Hash challenge	SHA-256	Fiat-Shamir
Hash keygen	SHA-256	Dérivation déterministe
Certificats CA	RSA-2048 / SHA-256 / PKCS#1 v1.5	Standard X.509-like

Limites pédagogiques

Ce projet est une **démonstration éducative**, pas un système de production. Les simplifications suivantes ont été faites intentionnellement :

Simplification	En production
<code>pki_store.json</code> en clair	Clé privée CA dans un HSM
DB en mémoire + JSON	Base de données persistante sécurisée
Pas de TLS	HTTPS obligatoire (sinon MitM actif possible)
Pas d'expiration de certificat	Durée de validité + révocation (CRL/OCSP)
$x = \text{SHA256}(\text{pwd}) \bmod Q$	KDF robuste (Argon2, scrypt) avec sel
Un seul niveau CA	Chaîne CA racine → CA intermédiaire → certificat

Références

- Schnorr, C.P. (1991). *Efficient signature generation by smart cards*. Journal of Cryptology.
- Fiat, A., Shamir, A. (1987). *How to prove yourself: practical solutions to identification and signature problems*.
- RFC 3526 — [More Modular Exponential \(MODP\) Diffie-Hellman groups for IKE](#)
- BIP-340 — [Schnorr Signatures for secp256k1](#)
- Boneh, D., Shoup, V. — [A Graduate Course in Applied Cryptography](#)

Auteur

Projet réalisé dans le cadre d'un cours de **cryptographie appliquée**.

Licence : MIT